UNIVERSIDAD DEL VALLE DE GUATEMALA

Ingeniera de Software 1 - Sección 30

Lynette García

Plataforma digital para la colaboración interdisciplinaria entre asociaciones estudiantiles universitarias

Angel Gabriel Sanabria Morales - 24725

Saul Esteban Castillo Arenas - 24915

Vernel Josué Hernández Cáceres - 24584

Derek Friedhelm Coronado Chilin - 24732

Samuel Antonio Robledo López - 241282

Guatemala, 2026

Resumen

El presente documento corresponde al tercer corte del proyecto del curso Ingeniería de Software 1, enfocado en la formalización del análisis y diseño el sistema propuesto: una plataforma digital centralizada para la publicación, colaboración y validación institucional de proyectos universitarios organizacionales e individuales. En los cortes anteriores se aplicó la metodología Design Thinking para identificar necesidades reales del entorno universitario, generar soluciones y validar prototipos.

En esta etapa se realiza la especificación formal de los requisitos funcionales y no funcionales del sistema, el modelado preliminar mediante diagramas UML, la definición de la persistencia de datos y la selección justificada de la arquitectura tecnológica. Asimismo, se documenta la planificación del proyecto y la gestión del equipo de desarrollo.

El objetivo principal de este corte es establecer una base estructurada y coherente que permita avanzar hacia la implementación del sistema, asegurando trazabilidad entre requisitos, modelo conceptual, diseño de base de datos y tecnologías avanzadas.

Introducción

La colaboración interdisciplinaria dentro del entorno universitario representa una oportunidad clave para fortalecer el aprendizaje práctico, fomentar el trabajo en equipo y generar impacto académico. Sin embargo, actualmente estos procesos se desarrollan de manera fragmentada y poco estructurada, dificultando la visibilidad de oportunidades, el seguimiento de trabajo y la validación institucional de la participación estudiantil.

En los cortes anteriores del proyecto se aplicaron las etapas iniciales de la metodología Design Thinking, identificando los principales actores del sistema, sus necesidades, problemáticas y oportunidades de mejora. Posteriormente, se desarrollaron prototipos preliminares y se definieron historias de usuario que describen el comportamiento esperado del sistema.

En este tercer corte se da un paso hacia la formalización técnica del proyecto. Se especifican los requisitos funcionales y no funcionales del sistema. Se elabora el modelo de clases preliminar, el diagrama entidad-relación de la base de datos y se justifica la selección de tecnologías que soportarán la solución propuesta. El propósito de esta etapa es consolidar un diseño coherente, escalable y técnicamente fundamentado que permita avanzar hacia la fase de implementación con una arquitectura sólida y bien definida.

El sistema está dirigido a toda la comunidad estudiantil de la UVG y soporta dos modalidades de publicación de proyectos: proyectos organizacionales, publicados por asociaciones, institutos, centros de investigación u otras organizaciones universitarias; y proyectos individuales, publicados por cualquier estudiante que necesite conformar un equipo de trabajo. Esta distinción es fundamental para el diseño del sistema y para la estructura de persistencia de datos que lo respalda.

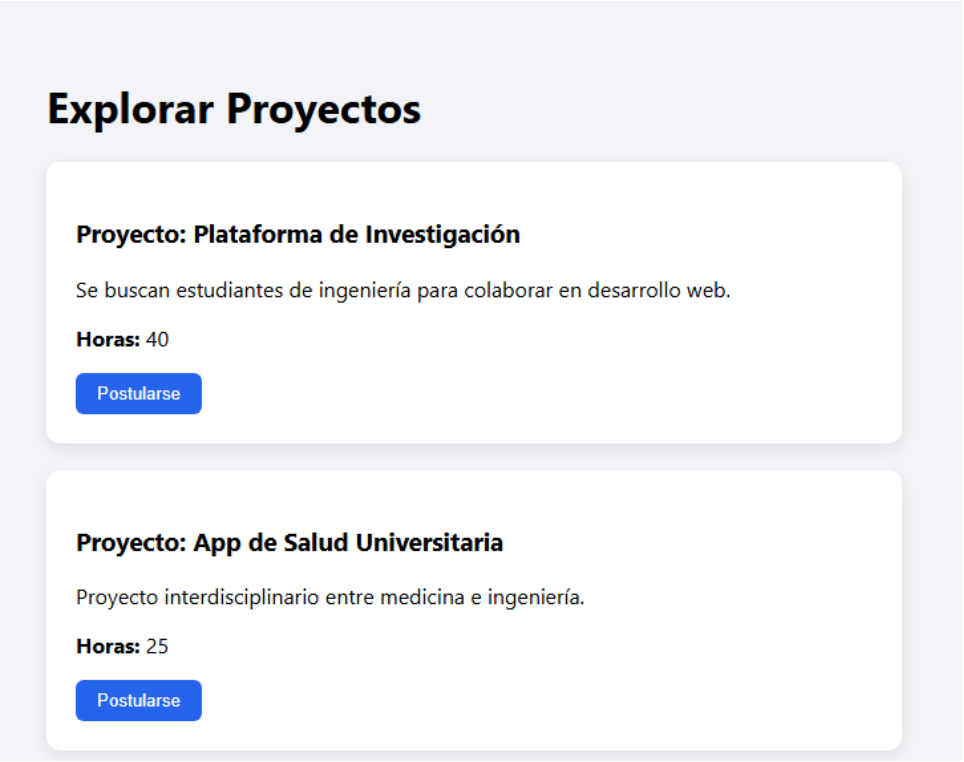
Design Thinking

Actualización de prototipos

En esta etapa del proyecto se realizaron mejoras progresivas al prototipo de la plataforma con base en el análisis de uso y en la retroalimentación obtenida durante las primeras validaciones con usuarios. El objetivo principal de estas iteraciones fue mejorar la claridad de la interfaz, facilitar la navegación entre los diferentes proyectos publicados y permitir un seguimiento más claro del progreso de los estudiantes dentro de cada proyecto.

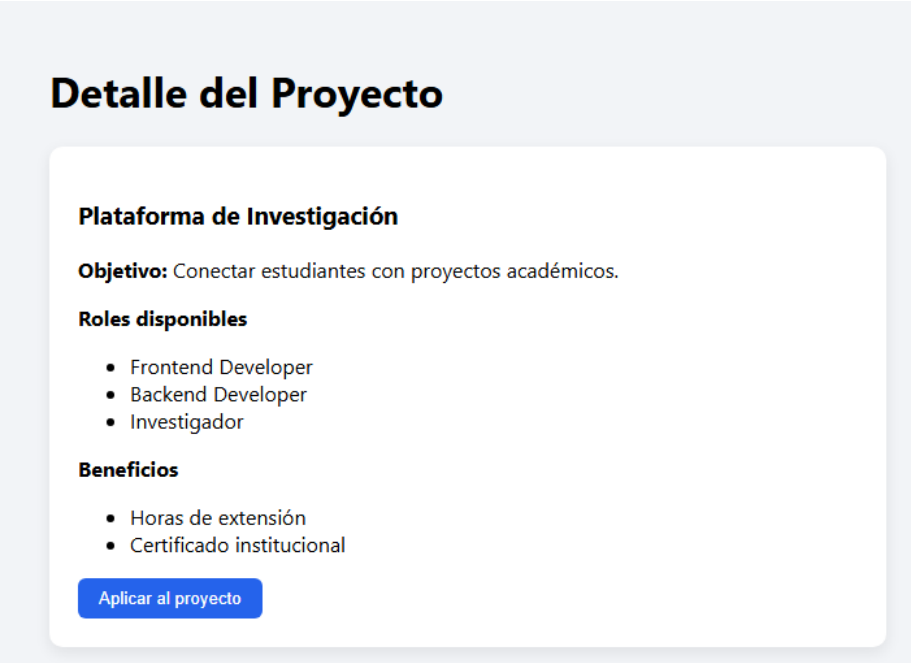
A lo largo del proceso, se generaron cuatro versiones principales del prototipo. Cada versión presentó avances en la organización de la información, en el modo de interactuar con la plataforma y en cómo se visualiza el progreso de las tareas.

El primer prototipo tuvo como objetivo mostrar la estructura general del sistema. Durante esta fase, se creó la interfaz para explorar proyectos, que permite a los alumnos observar múltiples opciones disponibles en la plataforma. En esta vista se presenta información esencial del proyecto, incluyendo su descripción y el número de horas académicas que brinda, y también un botón para comenzar el proceso de postulación.



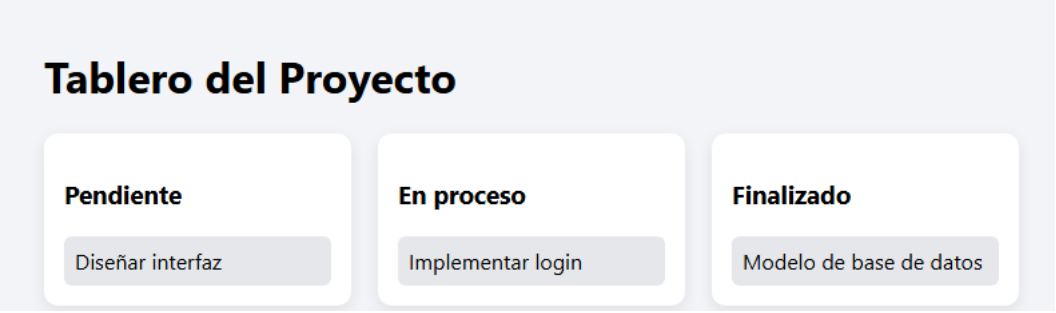
(Figura 1. Explorar proyectos)

En la segunda versión del prototipo se incorporó una vista más detallada de cada proyecto. Esta pantalla permite al usuario visualizar información más completa, como los objetivos del proyecto, Los roles disponibles para los estudiantes interesados y los beneficios académicos asociados a la participación. Esta mejora buscó que los usuarios comprendieran con mayor claridad qué implica cada proyecto antes de postularse.



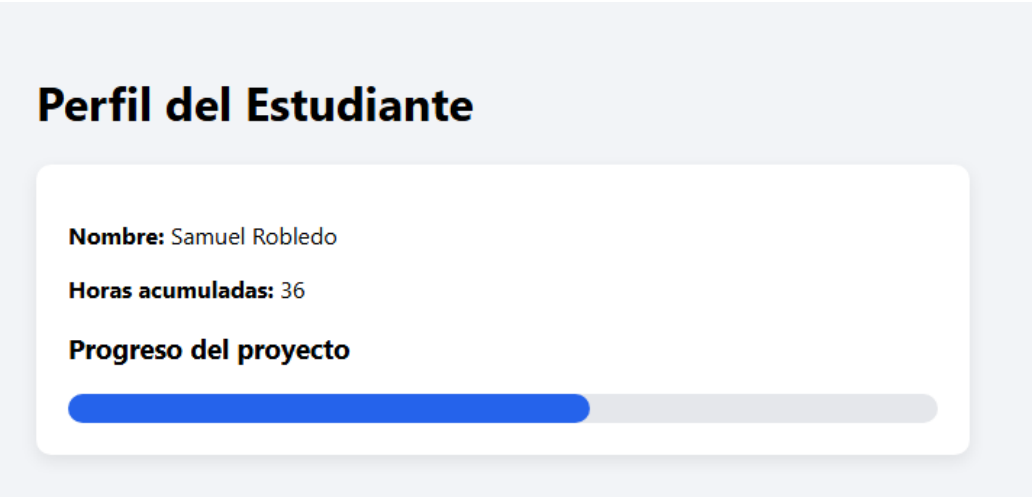
(Figura 2. Detalle del proyecto)

Después, se llevó a cabo una tercera iteración enfocada en la organización del trabajo de los proyectos. Esta versión incluye un tablero de tareas con una estructura similar a Kanban, que facilita la visualización del estado de cada tarea en el proyecto. Así, los alumnos pueden determinar con facilidad qué tareas están pendientes, cuáles se están realizando y cuáles ya han sido completadas.



(Figura 3. Tablero del proyecto)

Por último, se incluyó una vista de perfil del estudiante que muestra el avance individual en la plataforma. La cantidad de horas acumuladas y una barra de progreso que muestra el progreso de los alumnos en el proyecto se pueden ver en esta pantalla. Este elemento tiene como objetivo ofrecer una retroalimentación visual precisa de la participación y el rendimiento del usuario. Esta vista no es la del perfil del estudiante como tal sino que seria la vista del coordinador del proyecto y ver las estadisticas de cada estudiante.



(Figura 4. Progreso del estudiante)

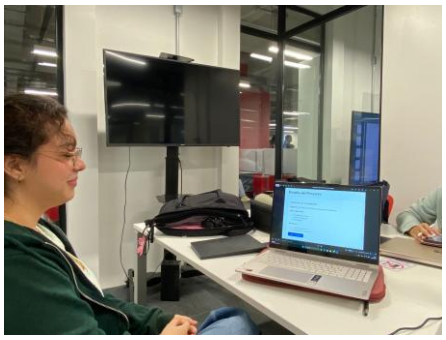
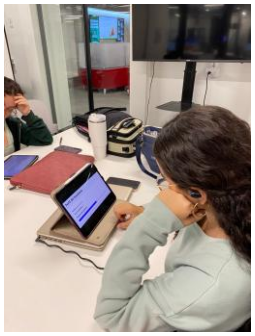
Evidencias de validación con usuarios

Con el propósito de validar el diseño del prototipo, se llevaron a cabo ensayos informales con alumnos universitarios que encarnan algunos de los perfiles detectados en el sistema, sobre todo, aquellos interesados en involucrarse en proyectos académicos, miembros de organizaciones universitarias y estudiantes que podrían publicar proyectos individuales.

Se mostró a los participantes las distintas pantallas del prototipo durante estas sesiones y se les solicitó que explicaran cómo emplearían la plataforma para investigar proyectos, examinar datos de un proyecto y entender el progreso de las actividades en el sistema. Este ejercicio ayudó a determinar si la estructura de navegación era clara y si se comprendía el objetivo de cada sección.

En términos generales, los usuarios señalaron que una plataforma centralizada para la publicación de proyectos universitarios es beneficiosa, porque hoy en día muchas oportunidades se distribuyen solo por medio de conexiones personales o redes informales. Además, indicaron que la vista de exploración de proyectos hace más fácil detectar con rapidez las oportunidades disponibles.

No obstante, algunos de los participantes indicaron que era esencial tener un método visual para comprender el avance del trabajo dentro de los proyectos. Se optó por incorporar un tablero de tareas estilo Kanban y una barra de progreso al perfil del alumno a partir de esta observación, lo que posibilita una visualización más nítida del estado de las actividades y del progreso individual.

Lista de cambios realizados

Versión del prototipo	**Cambio** realizado.	**Motivo del cambio**
Versión 1	Creación de la pantalla de exploración de proyectos	Permitir a los estudiantes visualizar proyectos disponibles dentro de la plataforma.
Versión 2	Incorporación de la vista de detalle del proyecto	Mostrar información más completa sobre objetivos, roles y beneficios.
Versión 3	Implementación de un tablero de tareas tipo Kanban	Facilitar el seguimiento del trabajo dentro del proyecto.
Versión 4	**Inclusión de una vista de seguimiento del progreso del estudiante y una vista administrativa de monitoreo para coordinadores.**	Mostrar visualmente el avance del usuario dentro del proyecto.

Cambios en requisitos funcionales

Con base en la retroalimentación que se recibió durante la validación de los prototipos, se hicieron ciertos cambios en las especificaciones funcionales del sistema, sobre todo en las vinculadas con el monitoreo de la labor y el análisis del avance.

Uno de los cambios más significativos fue enfatizar la importancia de tener un tablero de tareas que permita ver cómo están las actividades en cada proyecto. Esta función facilita que los alumnos entiendan con mayor claridad cómo se estructura el trabajo y qué deberes tienen que realizar.

Además, se consideró esencial incorporar mecanismos de retroalimentación visual para el seguimiento del progreso estudiantil, como la visualización de las horas acumuladas y el avance en el proyecto, complementados por una vista administrativa para el coordinador académico. Esto posibilita que los usuarios cuenten con una referencia nítida sobre su participación y el reconocimiento académico que recibirán.

Estas modificaciones garantizan que los requerimientos funcionales del sistema representen con mayor exactitud las necesidades que se detectaron durante las pruebas con usuarios y ayudan a optimizar la experiencia de uso de la plataforma en general.

Análisis del sistema

Historias de usuario (HU)

Estudiante interesado:

- **HU-1:** Como estudiante universitario que no pertenece necesariamente a una asociación, deseo acceder a una plataforma centralizada que publique todos los proyectos extracurriculares y académicos disponibles, para no depender de redes informales o del boca a boca para enterarme de oportunidades.
- **HU-2:** Como estudiante, deseo que todos los proyectos estén disponibles de manera equitativa para cualquier alumno de la universidad, sin importar su pertenencia a asociaciones o círculos específicos, para garantizar igualdad de acceso a oportunidades.
- **HU-3:** Como estudiante, deseo visualizar claramente en cada proyecto los beneficios académicos ofrecidos (horas beca, horas de extensión, certificados u otros reconocimientos), para decidir informadamente si me conviene postularme.
- **HU-4:** Como estudiante, deseo poder filtrar los proyectos por carrera, habilidades requeridas, tipo de proyecto y tipo de reconocimiento académico, para encontrar oportunidades alineadas con mi perfil e intereses.
- **HU-5:** Como estudiante, deseo postularme a un proyecto mediante el envío de mi perfil y una breve justificación, para expresar formalmente mi interés en participar.

Estudiante colaborador activo:

- **HU-6:** Como estudiante que participa en un proyecto, deseo que las metas, roles y expectativas estén claramente definidos desde el inicio, para evitar confusión y desmotivación.
- **HU-7: C**omo estudiante colaborador, deseo contar con un tablero visual donde pueda ver mis tareas asignadas y su estado (pendiente, en proceso, en revisión y finalizada), para dar seguimiento claro a mi trabajo.
- **HU-8:** Como estudiante colaborador, deseo recibir notificaciones sobre eventos relevantes del proyecto, como resolución de postulaciones, asignación de tareas, revisión de evidencias, validación de horas y emisión de certificados, para mantenerme informado del avance del proyecto sin depender de comunicación informal.
- **HU-9:** Como estudiante colaborador, deseo poder cargar evidencias digitales de mi trabajo (documentos, enlaces, reportes), para que mi participación pueda ser validada institucionalmente.
- **HU-10:** Como estudiante colaborador, deseo visualizar mi progreso individual y las horas acumuladas dentro de mi perfil, para tener claridad sobre mi avance y reconocimiento académico.

Líder de Asociación:

- HU-11: Como líder de una organización universitaria o como estudiante creador de un proyecto individual, deseo publicar proyectos en un espacio oficial y estandarizado dentro de la plataforma, incluyendo descripción, objetivos, roles requeridos y beneficios académicos, para garantizar visibilidad institucional y reclutar colaboradores.
- **HU-12:** Como líder de proyecto, deseo gestionar las postulaciones recibidas (aceptar o rechazar candidatos), para conformar un equipo adecuado según las necesidades del proyecto.

- **HU-13:** Como líder, deseo asignar tareas con fechas límite y definir formalmente los roles antes de iniciar el proyecto, para asegurar organización y claridad en el equipo.
- **HU-14:** Como líder, deseo supervisar el avance del proyecto mediante hitos, estado de tareas y evidencias cargadas, para evitar desorganización y abandono.

Coordinador Académico:

- **HU-15:** Como coordinador académico o investigador, deseo monitorear el avance general del proyecto mediante el seguimiento de hitos, evidencias revisadas y horas reportadas/aprobadas, para dar seguimiento continuo y no únicamente al final.
- **HU-16:** Como coordinador académico, deseo revisar y validar las evidencias cargadas por los estudiantes, para certificar formalmente su participación.
- **HU-17:** Como coordinador académico, deseo que el sistema genere certificados oficiales descargables una vez validado el trabajo, para formalizar el reconocimiento institucional.

Requisitos funcionales (RF)

Gestión de acceso y perfil:

- RF-1: El sistema debe permitir autenticación mediante credenciales institucionales.
    - Derivado de HU-1
- RF-2: El sistema debe mostrar un feed centralizado con todos los proyectos publicados.
    - Derivado de HU-1 y HU-2
- RF-3: El sistema debe permitir filtrar proyectos por carrera, habilidades, tipo de proyecto y tipo de reconocimiento.
    - Derivado de HU-4
- RF-4: El sistema debe mostrar en cada proyecto una ficha estandarizada que incluya objetivos, roles requeridos, habilidades necesarias, beneficios académicos y estado del proyecto.
    - Derivado de HU-3
- RF-5: El sistema debe permitir al estudiante enviar una postulación adjuntando su perfil y justificación.
    - Derivado de HU-5

Colaboración y seguimiento

- RF-6: El sistema debe requerir la asignación formal de roles antes de iniciar un proyecto.
    - Derivado de HU-6
- RF-7: El sistema debe proporcionar un tablero tipo Kanban con estados de tareas (por_hacer, en_progreso, en_revision, hecho).
    - Derivado de HU-7
- RF-8: El sistema debe generar y mostrar notificaciones sobre eventos relevantes del proyecto, incluyendo resolución de postulaciones, asignación de tareas, revisión de evidencias, validación de horas y emisión de certificados.
    - Derivado de HU-8
- RF-9: El sistema debe permitir cargar evidencias digitales asociadas a tareas o proyectos.
    - Derivado de HU-9
- RF-10: El sistema debe mostrar en el perfil del estudiante el progreso individual y las horas acumuladas.
    - Derivado de HU-10

Gestión de proyectos

- RF-11:
- El sistema debe permitir publicar proyectos con información estructurada, ya sea en nombre de una organización universitaria o como proyecto individual creado por un estudiante.
    - Derivado de HU-11
- RF-12: El sistema debe permitir al líder aceptar o rechazar postulaciones.
    - Derivado de HU-12
- RF-13: El sistema debe permitir asignar tareas con fechas límite y responsables definidos.
    - Derivado de HU-13
- RF-14: El sistema debe permitir consultar el avance del proyecto mediante hitos, estado de tareas, evidencias registradas y horas reportadas/aprobadas, sin generación automática de reportes institucionales en esta primera versión. El módulo de validación se limitará a revisión de evidencias, validación de horas y emisión de certificados.
    - Derivado de HU-14 y HU-15

Validación institucional

- RF-15: El sistema debe permitir al coordinador revisar evidencias cargadas por estudiantes.
    - Derivado de HU-16
- RF-16: El sistema debe permitir validar formalmente las horas realizadas por los estudiantes.
    - Derivado de HU-16
- RF-17: El sistema debe generar certificados oficiales descargables tras la validación del proyecto.
    - Derivado de HU-17

Trazabilidad

Historia de usuario	**Requisito funcional**
HU-1	RF-1
HU-2	RF-1 y RF-2
HU-3	RF-4
HU-4	RF-3
HU-5	RF-5
HU-6	RF-6
HU-7	RF-7
HU-8	RF-8
HU-9	RF-9
HU-10	RF-10
HU-11	RF-11
HU-12	RF-12
HU-13	RF-13
HU-14	RF-14
HU-15	RF-14
HU-16	RF-15 y RF-16
HU-17	RF-17

Clases Preliminares

Clases de Entidad (Modelo)

#	Clase	Justificación	RF asociados
1	**Usuario**	Representa la cuenta base de cualquier persona registrada en el sistema. Gestiona credenciales de autenticación, datos de identificación y estado de la cuenta. Los perfiles de acceso se gestionan mediante RolAcceso, UsuarioRolAcceso, Permiso y RolPermiso.	RF-1, RF-3, RF-5, RF-10
2	**Organizacion**	Agrupa a líderes y vincula proyectos con la organización estudiantil que los publica.	RF-11
3	**Proyecto**	Entidad principal del sistema. Contiene la información de convocatoria, estado, reconocimiento y estructura de trabajo.	RF-2, RF-4, RF-11, RF-14
4	**RolProyecto**	Define los roles requeridos dentro de un proyecto (habilidades, cantidad, descripción), permitiendo la asignación formal antes de iniciar.	RF-6
5	**Postulacion**	Registra la solicitud de un estudiante para participar en un proyecto a través de un rol específico, incluyendo justificación y estado de aprobación.	RF-5, RF-12
6	**Tarea**	Unidad de trabajo asignable dentro de un proyecto. Soporta el tablero Kanban con estados y fechas límite.	RF-7, RF-13
7	**Hito**	Punto de control dentro del proyecto que agrupa tareas y permite medir avance por fases.	RF-14
8	**Evidencia**	Archivo o enlace cargado por un estudiante como prueba de trabajo realizado en una tarea.	RF-9, RF-15
9	**RevisionEvidencia**	Registro de la evaluación formal de una evidencia por parte del coordinador, incluyendo resultado, comentario técnico y fecha de revisión.	RF-15, RF-16
10	**HorasParticipacion**	Registro formal de horas reportadas y aprobadas para una participación específica. Gestiona el flujo de validación: pendiente, aprobada o rechazada.	RF-15, RF-16

11	**Certificado**	Documento oficial generado tras la validación, con datos del proyecto, horas y responsable.	RF-17
12	**Notificacion**	Alerta dirigida a un usuario sobre eventos relevantes del sistema (postulación resuelta, tarea asignada, evidencia revisada, horas validadas, certificado emitido y participación actualizada).	RF-8
13	**BitacoraAuditoria**	Registro inmutable de acciones críticas del sistema para trazabilidad y seguridad.	RNF-Seguridad

El diagrama de clases de análisis presentado en esta sección es una vista antes que se ideara la base de datos, estas clases preliminares son las entidades principales del dominio funcional. Se hizo así ya que son "clases preliminares" que obtuvimos en base a las historias de usuario y requisitos funcionales, de igual forma abajo adjuntaremos las entidades completas de nuestra base de datos, ya que no sabemos con certeza si era necesario.

Entidades completas en la base de datos (luego de idearla para el proyecto de bases de datos)

Entidad	Llaves	Nota
Usuario	id_usuario (PK), correo (CK+UNIQUE)	
UsuarioTelefono	id_usuario_telefono (PK)	FK: id_usuario
PerfilEstudiante	id_usuario (PK+FK), carne (CK+UNIQUE)	1:1 con Usuario
Carrera	id_carrera (PK)	
Habilidad	id_habilidad (PK)	
UsuarioHabilidad	id_usuario_habilidad (PK), UNIQUE(id_usuario, id_habilidad)	
Interes	id_interes (PK), nombre_interes (CK+UNIQUE)	
UsuarioInteres	id_usuario_interes (PK), UNIQUE(id_usuario, id_interes)	

Cualidad	id_cualidad (PK), nombre_cualidad (CK+UNIQUE)	
UsuarioCualidad	id_usuario_cualidad (PK), UNIQUE(id_usuario, id_cualidad)	
ExperienciaPrevia	id_experiencia (PK)	
Organizacion	id_organizacion (PK)	
UsuarioOrganizacion	id_usuario_organizacion (PK), UNIQUE(id_usuario, id_organizacion)	Membresía permanente
Proyecto	id_proyecto (PK)	id_organizacion NULLABLE
ProyectoInteres	id_proyecto_interes (PK), UNIQUE(id_proyecto, id_interes)	
RolProyecto	id_rol_proyecto (PK), CK: (id_proyecto, nombre_rol)	
RequisitoHabilidadRol	id_requisito_habilidad (PK), UNIQUE(id_rol_proyecto, id_habilidad)	
Postulacion	id_postulacion (PK)	
ParticipacionProyecto	id_participacion (PK)	Vinculación por proyecto
Hito	id_hito (PK), CK: (id_proyecto, orden)	
Tarea	id_tarea (PK)	

AsignacionTarea	id_asignacion (PK), UNIQUE(id_tarea, id_usuario)	
Evidencia	id_evidencia (PK)	
RevisionEvidencia	id_revision (PK)	
HorasParticipacion	id_registro_horas (PK)	
Certificado	id_certificado (PK), codigo_verificacion (CK+UNIQUE)	
Notificacion	id_notificacion (PK)	
RolAcceso	id_rol_acceso (PK), nombre_perfil (CK+UNIQUE)	
Permiso	id_permiso (PK), nombre_permiso (CK+UNIQUE)	
UsuarioRolAcceso	id_usuario_rol_acceso (PK), UNIQUE(id_usuario, id_rol_acceso)	
RolPermiso	id_rol_permiso (PK), UNIQUE(id_rol_acceso, id_permiso)	
BitacoraAuditoria	id_auditoria (PK)	
ConfiguracionSistema	clave (PK)	Único sin surrogado

Enumeraciones

Enumeración	Valores	Usada por
EstadoProyecto	Borrador, publicado, en_progreso, cerrado	Proyecto

EstadoPostulacion	Pendiente, Aceptada, Rechazada	Postulacion
EstadoTarea	Por_hacer, En_progreso, En_revision, Hecho	Tarea
EstadoHoras	Pendiente, Aprobada, Rechazada	HorasParticipacion
TipoEvidencia	Enlace, archivo, reporte	Evidencia
TipoNotificacion	Proyecto_publicado, Postulacion_resuelta, Tarea_asignada, Evidencia_revisada, Horas_validadas, Certificado_emitido, Participacion_actualizada	Notificacion
TipoProyecto	Academico_horas_beca, Academico_experiencia, Extracurricular_extension	Proyecto

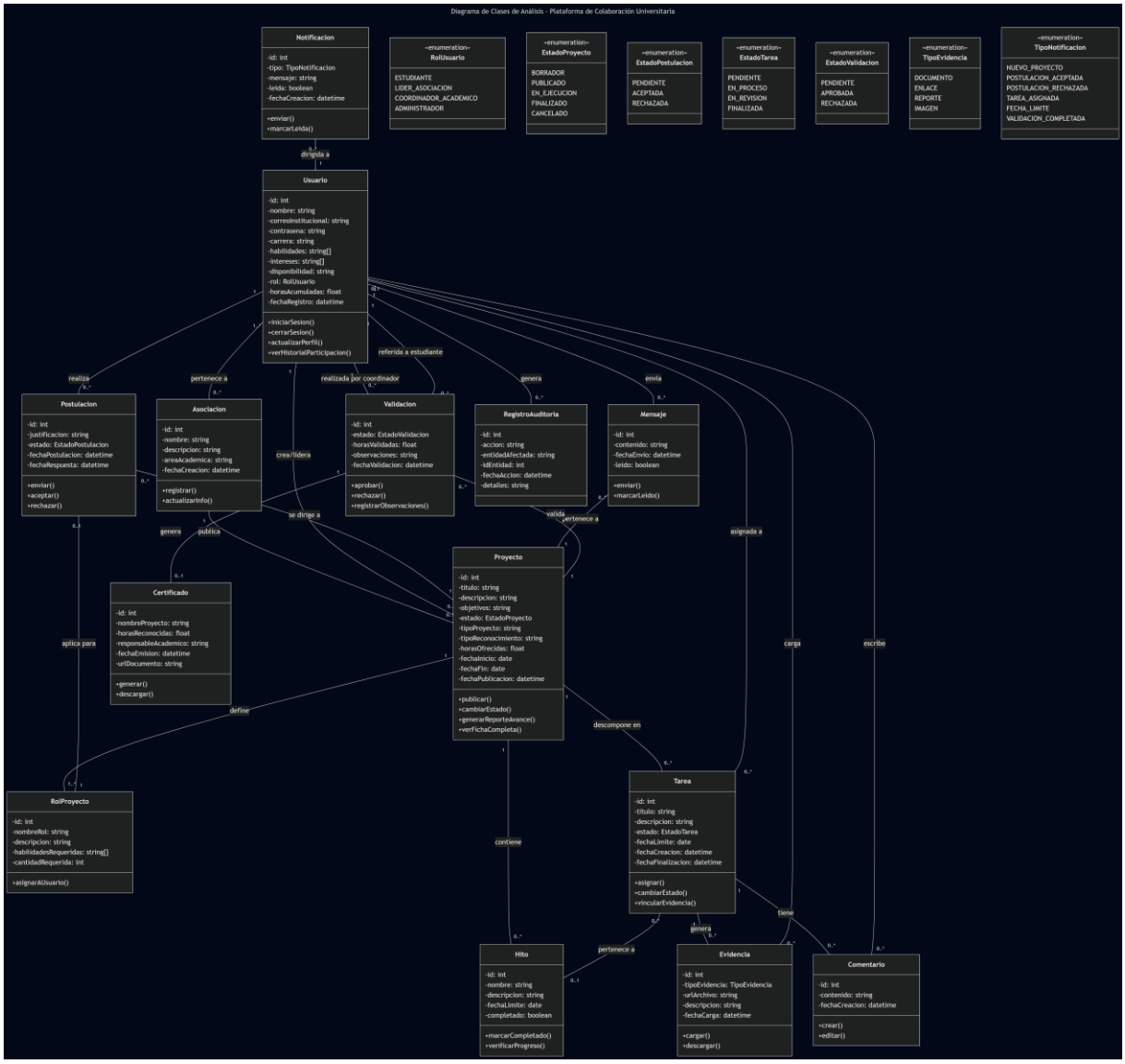
Relaciones principales

Relación	Tipo	Multiplicidad	Descripción
Usuario — Organizacion	Asociación	N:M	Relación N:M gestionada mediante la tabla intermedia UsuarioOrganizacion, que incluye rol_organizacion (propietario, administrador, miembro), fecha_ingreso y estado_membresia. La membresía es permanente e independiente de los proyectos.
Organizacion — Proyecto	Asociación	1:N / 0:1	Una organización puede publicar cero o más proyectos organizacionales. Un proyecto puede pertenecer a una organización o ser individual; si es individual, id_organizacion es NULL y creado_por es el responsable.
Usuario — Proyecto	Asociación	1:N	Un usuario puede crear uno o más proyectos. En proyectos individuales, el usuario creador es el responsable

			directo; en proyectos organizacionales, debe ser propietario o administrador de la organización publicadora.
Proyecto — RolProyecto	Composición	1:N	Cada proyecto define uno o más roles requeridos. Los roles no existen sin proyecto.
Proyecto — Hito	Composición	1:N	Un proyecto se estructura en hitos de avance.
Proyecto — Tarea	Composición	1:N	Un proyecto se descompone en múltiples tareas.
Usuario — Postulacion	Asociación	1:N	Un estudiante realiza postulaciones a roles de proyecto.
Postulacion — RolProyecto	Asociación	N:1	Cada postulación aplica para un rol específico. La vinculación al proyecto se obtiene transitivamente a través de RolProyecto.
Postulacion — ParticipacionProyecto	Asociación	1:0..1	Una postulación aceptada puede originar una participación.
RolProyecto — ParticipacionProyecto	Asociación	1:N	Un rol puede dar lugar a múltiples participaciones aceptadas.
Tarea — AsignacionTarea	Composición	N:1	Una tarea puede tener uno o más registros de asignación.
Tarea — Hito	Asociación	N:1	Una tarea puede pertenecer a un hito.
Tarea — Evidencia	Composición	1:N	Una tarea genera evidencias de trabajo.

AsignacionTarea — Usuario	Asociación	N:1	Cada registro de asignación vincula una tarea con un usuario específico.
Usuario — Evidencia	Asociación	1:N	Un usuario carga evidencias.
Notificacion — Usuario	Asociación	N:1	Las notificaciones se dirigen a un usuario.
Usuario — BitacoraAuditoria	Asociación	1:N	Las acciones del usuario se registran en auditoría.
Organizacion — UsuarioOrganizacion	Composición	1:N	Una organización tiene una o más membresías de usuarios.
Usuario — UsuarioOrganizacion	Asociación	1:N	Un usuario puede pertenecer a una o más organizaciones.
Evidencia — RevisionEvidencia	Asociación	1:N	Una evidencia puede tener una o más revisiones.
ParticipacionProyecto — HorasParticipacion	Asociación	1:N	Una participación puede generar uno o más registros de horas.
ParticipacionProyecto — Certificado	Asociación	1:0..N	Una participación validada puede generar certificados.

Diagrama de clases



Descripción de clases

- **Usuario:**
  - Representa a cualquier persona registrada en la plataforma. Gestiona la autenticación mediante credenciales institucionales y el mantenimiento del perfil base de la cuenta. Usuario representa la cuenta base del sistema. Los perfiles de acceso persistentes se gestionan mediante las entidades RolAcceso, UsuarioRolAcceso, Permiso y RolPermiso. Los actores ‘estudiante interesado’ y ‘estudiante colaborador activo’ representan estados de uso del estudiante dentro del sistema, no valores persistidos en un único atributo rol.
  - **Atributos principales:** id_usuario, correo, contrasena, nombre, apellido, foto_url, estado, fecha_creacion, fecha_ultima_sesion.
  - **Operaciones:** iniciarSesion(), cerrarSesion(), actualizarPerfil(), verHistorialParticipacion().

- **Organizacion:**
  - Entidad que puede publicar proyectos: asociación, instituto, grupo estudiantil, centro de investigación u otro (modelo estilo GitHub). Los miembros se gestionan en UsuarioOrganizacion y persisten independientemente de los proyectos.
  - **Atributos principales:** id_organizacion, nombre_organizacion, tipo_organizacion, descripcion_organizacion, correo_contacto, telefono_contacto, sitio_web, logo_url, estado_organizacion, creada_por, fecha_creacion.
  - **Operaciones:** registrar(), actualizarInfo().

- **Proyecto:**
  - Entidad central del sistema. Representa una convocatoria de colaboración publicada por una organización universitaria o por un estudiante de forma individual. Si el proyecto pertenece a una organización, se vincula mediante id_organizacion; si es individual, id_organizacion es NULL y creado_por identifica al responsable. Contiene toda la información estructurada de la ficha del proyecto: descripción, objetivos, tipo de reconocimiento, estado y fechas. Las horas del trabajo se modelan mediante horas_semanales_estimadas en RolProyecto y mediante HorasParticipacion / Certificado para el registro y validación formal de horas. Gestiona su ciclo de vida desde borrador, publicado y en_progreso hasta cerrado.
  - **Atributos principales:** id_proyecto, id_organizacion, titulo_proyecto, descripcion_proyecto, objetivos_proyecto, tipo_proyecto, estado_proyecto, modalidad_proyecto, ubicacion_proyecto, contexto_academico, url_recurso_externo, fecha_publicacion, fecha_inicio, fecha_fin_estimada, creado_por, fecha_creacion, fecha_actualizacion, eliminado_en.
  - **Operaciones:** publicar(), cambiarEstado(), actualizarInformacion(), verFichaCompleta().

- **RolProyecto:**
  - Define cada uno de los roles que requiere un proyecto antes de su ejecución. Especifica las habilidades requeridas, la cantidad de personas necesarias y una descripción del rol. Permite la asignación formal de estudiantes a funciones concretas, cumpliendo con el requisito RF-6.
  - **Atributos principales:** id, nombreRol, descripcion, habilidadesRequeridas, cantidadRequerida.
  - **Operaciones:** asignarAUsuario().

- **Postulacion:**
  - Solicitud formal de un estudiante para ocupar un rol específico.
  - **Atributos:** id_postulacion, id_usuario_postulante, id_rol_proyecto (NOT NULL), justificacion, estado_postulacion, fecha_postulacion, fecha_resolucion, resuelta_por, comentario_resolucion.
  - **Operaciones:** enviar(), aceptar(), rechazar().

- **Tarea:**
  - Unidad de trabajo dentro de un proyecto. Es la base del tablero Kanban y soporta los estados pendiente, en proceso, en revisión y finalizada. Puede vincularse a un hito y registrar asignaciones a través de la clase AsignacionTarea. Permite la carga de evidencias como insumo para la revisión formal del trabajo.
  - **Atributos principales:** id_tarea, id_proyecto, id_hito, titulo_tarea, descripcion_tarea, estado_tarea, prioridad, creada_por, fecha_creacion, fecha_limite, actualizada_en.
  - **Operaciones:** asignar(), cambiarEstado(), vincularEvidencia().

- **Hito:**
  - Punto de control dentro del ciclo de vida de un proyecto. Agrupa tareas relacionadas y permite medir el avance del proyecto por fases. Es utilizado tanto por líderes como por coordinadores para supervisar el progreso general.
  - **Atributos principales:** id, nombre, descripcion, fechaLimite, completado.
  - **Operaciones:** marcarCompletado(), verificarProgreso().

- **Evidencia:**
  - Archivo, enlace o reporte cargado por un estudiante como prueba de trabajo realizado en una tarea. Es el insumo principal para la validación institucional del trabajo estudiantil.
  - **Atributos principales:** id, tipoEvidencia, urlArchivo, descripcion, fechaCarga.
  - **Operaciones:** cargar(), descargar().

- **HorasParticipacion:**
  - Registro formal de horas reportadas y aprobadas para una participación específica. Gestiona el flujo completo de validación de horas: reporte por parte del estudiante y aprobación o rechazo por parte del coordinador académico.
  - **Atributos principales:** id_registro_horas, id_participacion, periodo_inicio, periodo_fin, horas_reportadas, horas_aprobadas, estado_horas, aprobado_por, fecha_aprobacion.
  - **Operaciones:** reportarHoras(), aprobarHoras(), rechazarHoras().

- **Certificado:**
  - Documento oficial emitido a partir de una participación validada.
  - **Atributos principales:** id_certificado, id_participacion, tipo_certificado, horas_certificadas, url_certificado, emitido_por, fecha_emision, codigo_verificacion.
  - Operaciones: generar(), descargar(), verificarCodigo().

- **Notificacion:**
  - Los tipos de notificación cubren eventos relevantes del sistema, como publicación de proyectos, resolución de postulaciones, asignación de tareas, revisión de evidencias, validación de horas, emisión de certificados y actualización de la participación.
  - **Atributos principales:** id_notificacion, id_usuario, tipo_notificacion, titulo_notificacion, mensaje_notificacion, datos_json, creada_en, leida_en.
  - **Operaciones:** enviar(), marcarLeida().

- **RegistroAuditoria:**
  - Registro inmutable de acciones críticas realizadas en el sistema. Almacena quién realizó la acción, qué entidad fue afectada, cuándo ocurrió, la dirección IP de origen y los detalles relevantes en formato JSON. Cumple con el requisito no funcional de seguridad y trazabilidad.
  - **Atributos principales:** id_auditoria, id_usuario (nullable), accion, tipo_objeto, id_objeto, detalle_json, ip_origen, fecha_evento.
  - **Operaciones:** (Sin operaciones públicas, los registros son creados automáticamente por el interceptor de auditoría.)

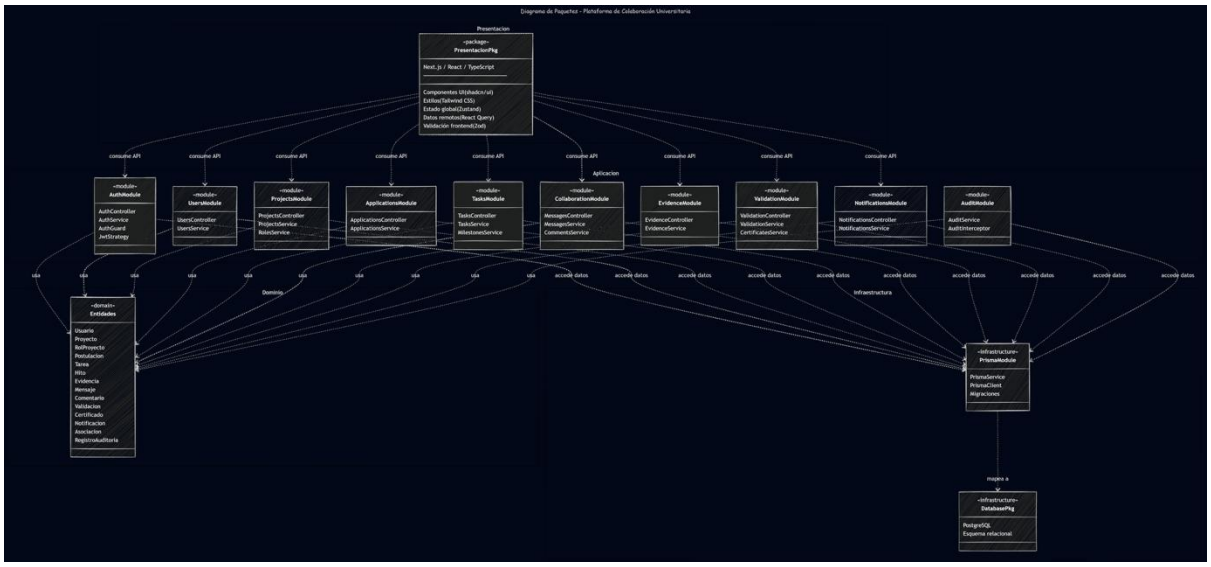
- **UsuarioOrganizacion:**
  - Relación asociativa que modela la membresía permanente de un usuario dentro de una organización.
  - **Atributos principales:** id_usuario_organizacion, id_usuario, id_organizacion, rol_organizacion, fecha_ingreso, estado_membresia.
  - **Operaciones:** invitarMiembro(), cambiarRol(), desactivarMembresia().

- **ParticipacionProyecto:**
  - Registro de la participación de un estudiante en un proyecto específico a través de un rol concreto. Es distinta de la membresía organizacional y puede existir aunque el estudiante no pertenezca a la organización publicadora.
  - **Atributos principales:** id_participacion, id_usuario, id_rol_proyecto, id_postulacion, estado_participacion, fecha_ingreso, fecha_salida.
  - **Operaciones:** activarParticipacion(), retirarParticipacion().

- **AsignacionTarea:**
  - Relación asociativa entre Tarea y Usuario que registra a quién se asignó una tarea, cuándo se asignó y quién realizó la asignación.
  - **Atributos principales:** id_asignacion, id_tarea, id_usuario, fecha_asignacion, asignado_por.
  - **Operaciones:** asignar(), reasignar().

- **RevisionEvidencia:**
  - Registro de la revisión de una evidencia cargada por un estudiante.
  - **Atributos principales:** id_revision, id_evidencia, id_revisor, resultado_revision, comentario, fecha_revision.
  - **Operaciones:** aprobarRevision(), rechazarRevision(), comentarRevision().

Diagrama de paquetes



Capa de Presentación (frontend/)

Responsable de la interfaz de usuario y la experiencia visual. Desarrollada con Next.js, React y TypeScript.

Componente	Responsabilidad
Pages	Vistas principales del sistema (inicio, explorar proyectos, detalle de proyecto, perfil, tablero, validación).
Components	Componentes UI reutilizables construidos con shadcn/ui y Tailwind CSS.
Store	Estado global del frontend gestionado con Zustand (sesión de usuario, filtros activos, etc.).
Hooks	Lógica de datos remotos mediante React Query (peticiones al backend, cacheo, revalidación).
Validators	Esquemas de validación Zod para formularios y datos de entrada.

Capa de Aplicación (backend/src/modules/)

Contiene la lógica de negocio organizada en módulos NestJS. Cada módulo agrupa un controlador (recibe peticiones HTTP), un servicio (ejecuta lógica de negocio) y las dependencias necesarias.

Módulo	Responsabilidad	RF cubiertos
auth	Autenticación con credenciales institucionales, gestión de sesión JWT, guards de autorización por rol.	RF-1
users	Gestión de perfiles de usuario: creación, edición, consulta de historial y horas acumuladas.	RF-10
projects	Publicación, edición, consulta y cambio de estado de proyectos. Gestión de roles de proyecto.	RF-2, RF-3, RF-4, RF-6, RF-11
applications	Envío, consulta y gestión de postulaciones (aceptar/rechazar).	RF-5, RF-12

tasks	Creación y asignación de tareas, gestión de hitos, cambio de estados del tablero Kanban.	RF-7, RF-13
evidence	Carga, consulta y descarga de evidencias digitales asociadas a tareas.	RF-9
validation	Revisión de evidencias, validación de horas y generación de certificados.	RF-14, RF-15, RF-16, RF-17
notifications	Envío y consulta de notificaciones automáticas por eventos del sistema.	Transversal
audit	Registro automático de acciones críticas mediante interceptor NestJS.	RNF-Seguridad

Capa de Dominio (backend/src/domain/)

Contiene las entidades del sistema y las reglas de negocio independientes de la infraestructura. No depende de ningún framework específico.

Componente	Contenido
entities/	Definición de las 14 entidades del dominio (Usuario, Proyecto, Tarea, etc.) con sus reglas de negocio.
enums/	Las 7 enumeraciones del sistema (RolUsuario, EstadoProyecto, etc.).

Capa de Infraestructura (backend/src/infrastructure/)

Gestiona la persistencia de datos y comunicación con la base de datos PostgreSQL a través de Prisma ORM.

Componente	Responsabilidad
prisma/	Módulo Prisma con PrismaService, esquema de base de datos y migraciones.
database/	Configuración de PostgreSQL y conexión.

- **auth**
  - Gestiona todo el proceso de autenticación y autorización del sistema. Incluye el controlador para login/logout, el servicio de generación y verificación de tokens JWT, los guards que protegen rutas por rol de usuario y la estrategia de validación de credenciales institucionales. Es el primer punto de contacto para cualquier usuario que acceda a la plataforma.
- users
  - Administra los perfiles de usuario del sistema. Permite la creación y edición de perfiles con información académica (carrera, habilidades, intereses, disponibilidad), la consulta del historial de participación en proyectos y la visualización de horas acumuladas. Sirve como módulo de soporte para otros módulos que requieren información de los usuarios.

- **projects**
  - Módulo central del sistema. Gestiona el ciclo de vida completo de los proyectos: desde su creación como borrador, publicación con ficha estandarizada, ejecución y seguimiento hasta su finalización o cancelación. Incluye también la gestión de roles de proyecto (definición de roles requeridos, habilidades y cantidad). Expone el feed centralizado con filtros por carrera, habilidades, tipo de proyecto y reconocimiento académico.
- **applications**
  - Gestiona el flujo de postulaciones. Permite a los estudiantes enviar solicitudes adjuntando su perfil y justificación, y a los líderes de proyecto aceptar o rechazar candidatos. Mantiene el estado de cada postulación y registra las fechas de envío y respuesta.
- **tasks**
  - Gestiona la estructura de trabajo dentro de los proyectos. Incluye la creación, asignación y seguimiento de tareas con estados tipo Kanban (pendiente, en proceso, en revisión, finalizada), así como la gestión de hitos que agrupan tareas y permiten medir el avance por fases. Genera la información necesaria para barras de progreso e indicadores de seguimiento del proyecto.
- **evidence**
  - Gestiona la carga, almacenamiento y consulta de evidencias digitales. Los estudiantes suben documentos, enlaces o reportes como prueba de trabajo realizado en sus tareas. Estas evidencias son el insumo principal para la validación institucional posterior.
- **validation**
  - Módulo de supervisión institucional. Permite a los coordinadores académicos revisar evidencias, validar formalmente las horas trabajadas y generar certificados oficiales descargables.
- **notifications**
  - Gestiona el sistema de alertas del sistema. Envía notificaciones automáticas a los usuarios cuando ocurren eventos relevantes: publicación de nuevos proyectos, cambio de estado de postulaciones, asignación de tareas, proximidad de fechas límite y finalización de validaciones.
- **audit**
  - Módulo transversal de seguridad. Registra automáticamente todas las acciones críticas del sistema mediante un interceptor NestJS. Cada registro incluye el usuario, la acción realizada, la entidad afectada, la fecha y los detalles relevantes. Cumple con el requisito no funcional de auditoría y trazabilidad.

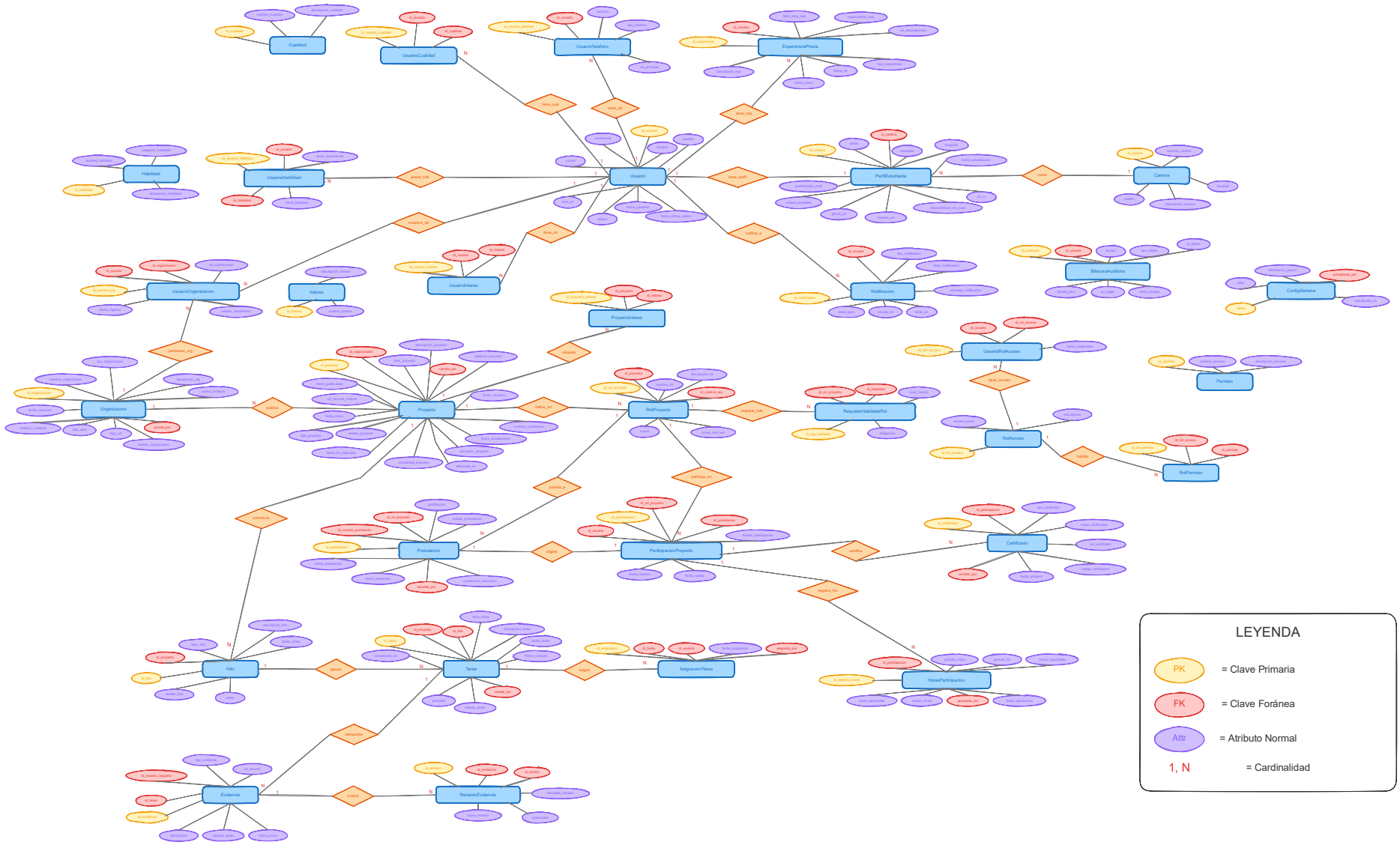
Relaciones entre Paquetes

- **Presentación y Aplicación:** El frontend consume los endpoints REST expuestos por los módulos del backend. No accede directamente a la base de datos.
- **Aplicación y Dominio:** Los servicios de cada módulo utilizan las entidades del dominio para aplicar reglas de negocio.
- **Aplicación y Infraestructura:** Los servicios acceden a datos mediante Prisma, sin ejecutar consultas SQL directas.
- **Dominio (independiente):** La capa de dominio no depende de ninguna otra capa. Define las reglas de negocio de forma pura.

Principios aplicados

- **Alta cohesión:** Cada módulo agrupa funcionalidades estrechamente relacionadas. Por ejemplo, el módulo tasks gestiona tareas y hitos, que están semánticamente vinculados.
- **Bajo acoplamiento:** Los módulos se comunican mediante interfaces definidas (servicios inyectados). Un cambio en evidence no afecta a projects.
- **Separación de responsabilidades:** La presentación no contiene lógica de negocio, la aplicación no conoce detalles de base de datos, y el dominio es independiente de frameworks.

Diagrama Entidad-Relación (DER)



Diseño del sistema

Requisitos no funcionales

Categoría	Requisito no funcional	Forma de medición
Apariencia / Interfaz externa	La interfaz deberá ser consistente en tipografía, colores, espaciado y componentes en todas las pantallas clave.	Checklist de 10 criterios aplicado a 6 pantallas clave. Cumple si cada pantalla satisface al menos 9 de los 10 criterios.
Apariencia / Interfaz externa	El menú principal y la navegación global deberán mantenerse idénticos en todas las vistas protegidas.	Revisión funcional de todas las rutas protegidas. Cumple si el menú aparece con las mismas opciones en el 100% de ellas.
Usabilidad	Al menos el 80% de usuarios nuevos deberá completar el flujo de exploración y postulación sin ayuda externa en menos de 10 minutos.	Prueba controlada con 10 participantes sin experiencia previa en la plataforma. Cumple si al menos 8 completan ambas tareas en el tiempo establecido sin asistencia.
Rendimiento	Las páginas principales (Inicio y Explorar proyectos) deberán obtener un puntaje de Performance mayor o igual a 80 en Lighthouse modo escritorio.	Medición con Lighthouse en modo escritorio desde Chrome. Se registra el puntaje de Performance para ambas páginas. Cumple si ambas alcanzan ≥ 80.
Rendimiento	El sistema deberá mantener una disponibilidad mayor o igual al 99% durante la ventana de evaluación de 14 días.	Monitor de disponibilidad activo durante 14 días consecutivos. Cumple si el reporte final indica uptime ≥ 99% (tiempo máximo de caída permitido: ≈ 2 horas 24 minutos).
Soporte	El sistema deberá permitir nuevos despliegues sin interrumpir las funcionalidades críticas.	Ejecución de un smoke test automático post-despliegue que verifica inicio de sesión, exploración de proyectos y envío de postulación. Cumple si los tres flujos responden sin error.
Soporte	Deberá existir documentación técnica (README y guía de instalación) y manual de usuario actualizados para la fecha de entrega.	Verificación de existencia y completitud de README, guía de instalación con comandos ejecutables y manual de usuario con capturas de pantalla. Cumple si están presentes y actualizados en la semana previa a la entrega.

Portabilidad	El sistema deberá ser compatible con las dos versiones más recientes de Chrome, Edge y Firefox en escritorio.	Ejecución de tareas críticas (login, exploración, postulación) en cada navegador y versión. Cumple si no se registra ningún fallo funcional en ninguna combinación.
Portabilidad	La interfaz deberá ser responsiva y operable en resoluciones de móvil (360px), tablet (768px) y escritorio (1366px).	Pruebas visuales y funcionales en los tres anchos usando DevTools. Cumple si no hay desbordamientos de contenido y el flujo completo de postulación se puede ejecutar en cada resolución.
Seguridad	Todo acceso a rutas protegidas deberá requerir autenticación activa; los intentos de acceso directo sin sesión deberán redirigir a inicio de sesión.	Intentos manuales de acceso directo a URLs protegidas sin sesión activa. Cumple si el 100% de los intentos redirige correctamente a la pantalla de inicio de sesión.
Seguridad	El sistema deberá registrar en la bitácora de auditoría cada acción crítica con usuario, tipo de acción, fecha/hora y objeto afectado.	Revisión de la tabla BitacoraAuditoria tras sesión de pruebas. Cumple si se encuentran registrados al menos 20 eventos con todos los campos requeridos correctamente poblados.
Confiabilidad	El sistema deberá poder restaurar la base de datos desde un respaldo en un tiempo máximo de 60 minutos.	Simulación de falla en entorno de pruebas: eliminación de base de datos y restauración desde respaldo, con medición del tiempo total. Cumple si el proceso completo tarda ≤ 60 minutos.
Confiabilidad	El sistema deberá generar respaldos automáticos diarios de la base de datos sin intervención manual.	Revisión del historial de respaldos después de 7 días de operación continua. Cumple si existen al menos 7 respaldos consecutivos completos con diferencia de ≤ 25 horas entre cada uno.
Ayudas y documentación en línea	Deberá existir un enlace visible de ayuda accesible desde cualquier pantalla protegida de la plataforma.	Revisión funcional de todas las vistas protegidas. Cumple si el enlace de ayuda aparece y es

		funcional en el 100% de las pantallas protegidas.
Restricciones de diseño e implementación	La arquitectura del sistema deberá separar claramente la capa de presentación, la lógica de negocio y el acceso a datos.	Revisión de estructura del repositorio y diagrama de capas. Cumple si se identifican módulos separados para frontend, backend y capa de datos sin dependencias cruzadas directas entre capa de presentación y base de datos.
Restricciones de diseño e implementación	El código fuente deberá mantenerse en un repositorio GitHub con historial de commits, ramas activas y evidencia de trabajo colaborativo.	Verificación del repositorio GitHub. Cumple si existen commits de al menos 4 integrantes, al menos 2 ramas activas y al menos 3 pull requests cerrados durante el ciclo de desarrollo.

Selección de tecnología

Enfoque tecnológico general

El sistema fue concebido como una aplicación web debido a la naturaleza de sus usuarios y al contexto universitario en el que operará. Durante la fase de empatía y validación realizada en los cortes anteriores, los perfiles identificados (estudiantes interesados, colaboradores activos, líderes de asociaciones y coordinadores académicos) manifestaron la necesidad de acceder a la plataforma desde distintos dispositivos y ubicaciones sin requerir instalación local.

El desarrollo web permite:

- Acceso multiplataforma (móvil, tablet, escritorio)
- Disponibilidad inmediata desde navegador
- Actualizaciones centralizadas
- Escalabilidad institucional

Además, las tecnologías web basadas en JavaScript continúan liderando el desarrollo moderno según la encuesta anual de desarrolladores de Stack Overflow 2025, lo cual respalda la decisión de utilizar un stack orientado a desarrollo web moderno (Stack Overflow, 2025).

Por estas razones, se evaluaron tecnologías enfocadas específicamente en el desarrollo de aplicaciones web modernas.

Las tecnologías adoptadas para el desarrollo del sistema son las siguientes:

- Next.js: Framework frontend basado en React
- React: Biblioteca para construcción de interfaces de usuario
- TypeScript: Lenguaje tipado sobre JavaScript
- NestJS: Framework backend basado en Node.js

- Prisma: ORM para acceso seguro y tipado a base de datos (Object Relational Mapping, lo que nos permite usar objetos en lugar de escribir SQL manual)
- PostgreSQL: Sistema de gestión de base de datos relacional
- Tailwind CSS: Framework CSS utility-first (lo que nos ayudara a construir estilos usando clases pequeñas reutilizables en lugar de escribir CSS desde cero)
- shadcn/ui: Componentes reutilizables para interfaz
- React Query (TanStack Query): Manejo de datos asincrónicos
- Zustand: Manejo de estado global ligero
- Zod: Validación estructurada de datos

Frontend: Justificación del Uso de Next.js y React con TypeScript

La plataforma requiere una interfaz dinámica que permita interacción constante entre estudiantes, líderes de asociaciones y coordinadores académicos. React fue seleccionado como base del frontend debido a su liderazgo en el ecosistema global de desarrollo web, posicionándose como una de las tecnologías más utilizadas por desarrolladores profesionales (Stack Overflow, 2025).

Next.js fue seleccionado como framework principal del frontend debido a su capacidad de renderizado híbrido (SSR y CSR), optimización automática y estructura basada en rutas, lo que mejora el rendimiento y la experiencia del usuario. Asimismo, Next.js es recomendado dentro del ecosistema React para aplicaciones que requieren capacidades avanzadas de servidor y optimización de producción (Vercel, s. f.).

React permite la construcción de componentes reutilizables, favoreciendo la mantenibilidad del sistema. TypeScript aporta tipado estático, reduciendo errores en tiempo de desarrollo y mejorando la escalabilidad del proyecto al trabajar en equipo.

Justificación del uso de NestJS

NestJS fue seleccionado como framework backend debido a su arquitectura modular y su compatibilidad nativa con TypeScript. Permite organizar el sistema en módulos independientes (usuarios, proyectos, tareas, validaciones), promoviendo alta cohesión y bajo acoplamiento.

Su sistema de inyección de dependencias y separación en capas facilita la mantenibilidad y escalabilidad futura del sistema. Además, NestJS se ha consolidado como uno de los frameworks más adoptados dentro del ecosistema Node.js para aplicaciones empresariales modernas (GitHub, 2024).

Uso de librerias frontend.

- **Tailwind CSS:** Ya que es un utility-first CSS framework que permite construir interfaces usando clases atómicas de estilo, reduciendo la necesidad de CSS personalizado y acelerando la maquetación de componentes visuales. Esta librería facilita la construcción de diseños responsivos y consistentes con bajo esfuerzo de mantenimiento, lo que contribuye al cumplimiento de los requisitos de apariencia e interfaz del sistema. Su adopción ha crecido significativamente dentro del desarrollo frontend moderno (npm trends, 2024).

- **shadcn/ui:** Ya que es una colección de componentes UI construidos sobre Radix UI y Tailwind CSS, que ofrece componentes listos para uso (botones, formularios, menús, etc.) con un diseño coherente y accesible. Esta librería agiliza el desarrollo de interfaces reutilizables y bien estructuradas, lo cual es crucial para mantener consistencia visual en todas las vistas.

- **React Query (TanStack Query):** Ya que será empleada para el manejo eficiente de la interacción con APIs, gestión de peticiones HTTP y sincronización de datos asíncronos. Esta librería permite cacheo automático, revalidación de datos, manejo de estados de carga y errores sin la necesidad de escribir lógica compleja, lo que reduce significativamente el código boilerplate requerido para manejar datos remotos. React Query es ampliamente adoptada en proyectos modernos de React para manejo de estado remoto (TanStack, 2024).

- **Zustand:** Ya que será utilizado para el manejo de estado global ligero en el frontend. A diferencia de soluciones más complejas como Redux, Zustand proporciona una API minimalista y eficiente, ideal para aplicaciones con estados compartidos simples o medianamente complejos. Su crecimiento sostenido y volumen de descargas reflejan su adopción creciente en aplicaciones modernas (npm trends, 2024).

- **Zod:** Ya que será la librería principal para validaciones de esquemas de datos tanto en frontend como backend. Permite definir reglas estructuradas y consistentes para validación de objetos y datos, integrándose de forma fluida con TypeScript, lo que incrementa la seguridad y robustez de las validaciones de entrada del usuario. Zod es actualmente una de las librerías de validación más utilizadas dentro del ecosistema TypeScript (npm trends, 2024).

Las librerías mencionadas constituyen el núcleo tecnológico del frontend. Sin embargo, durante la implementación podrán incorporarse dependencias auxiliares orientadas a optimizar experiencia de usuario, validaciones específicas o mejoras de rendimiento. La selección de dichas dependencias seguirá principios de arquitectura limpia, minimización de complejidad y control del tamaño del bundle final de producción. Esto permite mantener una aplicación escalable sin comprometer mantenibilidad ni desempeño.

Comparación de tecnologías frontend: Next.js (React) vs Angular vs Nuxt (Vue.js) vs SvelteKit

Para la capa de presentación del sistema se evaluaron cuatro alternativas de frameworks frontend ampliamente utilizados en el desarrollo de aplicaciones web modernas. La evaluación se realizó considerando las necesidades específicas del proyecto: construcción de interfaces dinámicas con componentes reutilizables, renderizado eficiente, compatibilidad con TypeScript y disponibilidad de un ecosistema maduro de librerías complementarias.

Next.js (React)

Next.js es un framework basado en React que ofrece renderizado híbrido (SSR y CSR), enrutamiento basado en sistema de archivos y optimización automática de producción. React, como biblioteca base, permite la construcción de interfaces mediante componentes reutilizables y cuenta con el ecosistema más amplio dentro del desarrollo frontend moderno.

Ventajas orientadas al proyecto: Next.js permite que las páginas públicas del sistema, como la exploración de proyectos y las fichas de detalle, se rendericen del lado del servidor, lo cual mejora el tiempo de carga inicial y contribuye al cumplimiento del requisito no funcional de rendimiento establecido en la métrica de Lighthouse. Su ecosistema es compatible de forma nativa con las librerías seleccionadas para el proyecto, incluyendo Tailwind CSS, shadcn/ui, React Query y Zustand. Además, la amplia documentación disponible y la comunidad activa facilitan la resolución de problemas durante

el desarrollo, aspecto relevante considerando que el equipo opera dentro de un contexto académico con tiempos limitados.

Desventajas orientadas al proyecto: La curva de aprendizaje inicial puede ser moderada para miembros del equipo que no estén familiarizados con el modelo de renderizado híbrido de Next.js. Asimismo, la configuración de despliegue puede requerir mayor atención en comparación con frameworks que generan únicamente archivos estáticos.

Angular

Angular es un framework completo desarrollado por Google que incluye de forma integrada manejo de estado, enrutamiento, formularios reactivos e inyección de dependencias. Utiliza TypeScript de forma obligatoria.

Ventajas orientadas al proyecto: Angular ofrece una arquitectura modular estricta que podría facilitar la organización del código en un sistema con múltiples vistas y roles de usuario como el propuesto. Su sistema de formularios reactivos sería útil para las pantallas de postulación y creación de proyectos. Al incluir TypeScript de forma obligatoria, garantiza tipado fuerte en todo el código frontend.

Desventajas orientadas al proyecto: Angular presenta una curva de aprendizaje considerablemente más pronunciada que React, lo cual representaría un riesgo significativo para el equipo dado el tiempo limitado de desarrollo disponible en el semestre. El peso del bundle de producción tiende a ser mayor, lo que podría dificultar el cumplimiento del requisito de rendimiento en Lighthouse. Además, el ecosistema de componentes UI reutilizables de Angular es menos extenso que el de React, lo que obligaría al equipo a invertir más tiempo en la construcción de la interfaz visual.

Nuxt (Vue.js)

Nuxt es un framework basado en Vue.js que ofrece capacidades similares a Next.js, incluyendo renderizado del lado del servidor, enrutamiento automático y generación estática. Vue.js se caracteriza por su simplicidad y su sistema de reactividad intuitivo.

Ventajas orientadas al proyecto: Vue.js posee una curva de aprendizaje más suave que React en sus conceptos fundamentales, lo que podría facilitar la incorporación rápida de miembros del equipo. Nuxt ofrece funcionalidades de renderizado híbrido comparables a las de Next.js, permitiendo optimizar el rendimiento de las páginas públicas del sistema.

Desventajas orientadas al proyecto: El ecosistema de Vue.js, aunque creciente, es considerablemente menor que el de React en términos de librerías complementarias, componentes UI y recursos de aprendizaje disponibles. La disponibilidad de herramientas equivalentes a shadcn/ui, React Query y Zustand dentro del ecosistema de Vue es más limitada, lo que requeriría buscar alternativas con menor nivel de madurez y documentación. Asimismo, la adopción profesional de Vue.js se encuentra por debajo de la de React según las encuestas recientes de desarrolladores (Stack Overflow, 2025).

SvelteKit

SvelteKit es un framework basado en Svelte que compila los componentes a código JavaScript nativo en tiempo de construcción, eliminando la necesidad de un runtime en el navegador. Ofrece renderizado del lado del servidor y enrutamiento basado en archivos.

Ventajas orientadas al proyecto: SvelteKit genera bundles de producción más pequeños que los de React y Vue, lo cual podría contribuir significativamente al rendimiento del sistema. Su sintaxis es concisa y reduce la cantidad de código necesario para implementar componentes interactivos como el tablero Kanban o las barras de progreso.

Desventajas orientadas al proyecto: El ecosistema de Svelte es significativamente más reducido que el de React. No existen equivalentes maduros para librerías como shadcn/ui, React Query o Zustand, lo que obligaría al equipo a desarrollar soluciones propias o utilizar alternativas con menor soporte comunitario. La comunidad y la documentación disponible son considerablemente menores, lo que incrementa el riesgo de encontrar problemas sin solución documentada durante el desarrollo. Además, la adopción profesional de Svelte permanece por debajo del 5% según las encuestas más recientes (Stack Overflow, 2025), lo que limita las referencias y ejemplos de implementación disponibles.

Tras el análisis comparativo, se seleccionó Next.js con React y TypeScript como stack de presentación del sistema. Esta decisión se fundamenta en la amplitud de su ecosistema, la compatibilidad nativa con todas las librerías complementarias requeridas por el proyecto, las capacidades de renderizado híbrido que contribuyen al cumplimiento de los requisitos no funcionales de rendimiento, y la extensa disponibilidad de documentación y recursos comunitarios que minimizan el riesgo de bloqueos durante el desarrollo. Adicionalmente, React lidera consistentemente las métricas de adopción profesional dentro del desarrollo frontend moderno (Stack Overflow, 2025), lo que facilita la transferencia de conocimiento y la mantenibilidad futura del sistema.

Backend: Justificación del uso de Prisma

Para la capa de acceso a datos se utilizará Prisma como ORM (Object-Relational Mapping) principal para interactuar con PostgreSQL.

Prisma fue seleccionado debido a su integración nativa con TypeScript, lo que permite generar tipos automáticamente a partir del esquema de base de datos, reduciendo errores en tiempo de desarrollo y garantizando coherencia entre el modelo de datos y la lógica de negocio.

Entre las razones principales para su elección se encuentran:

- Generación automática de tipos TypeScript.
- Migraciones controladas y versionadas de base de datos.
- Consultas SQL optimizadas con sintaxis clara y segura.
- Integración sencilla con NestJS.
- Prevención de errores comunes de consultas manuales.

El ecosistema Node.js y TypeScript continúa liderando el desarrollo moderno (Stack Overflow, 2025), lo que respalda el uso de herramientas como Prisma dentro de este stack tecnológico.

Comparación de frameworks backend: NestJS vs Express.js vs Fastify vs Spring Boot

Para la capa de aplicación del sistema se evaluaron cuatro alternativas de frameworks backend considerando los siguientes criterios: capacidad de estructuración modular del código, compatibilidad con TypeScript, sistema de inyección de dependencias, soporte para autenticación y autorización basada en roles, y madurez del ecosistema.

NestJS

NestJS es un framework para Node.js construido sobre TypeScript que implementa una arquitectura modular inspirada en Angular. Ofrece inyección de dependencias nativa, un sistema de módulos bien definido y soporte integrado para guards de autorización, interceptores y middleware.

Ventajas orientadas al proyecto: La arquitectura modular de NestJS permite organizar el sistema en módulos independientes que reflejan directamente los dominios funcionales del proyecto: autenticación, usuarios, proyectos, postulaciones, tareas, colaboración, evidencias, validación, notificaciones y auditoría. Su sistema de guards facilita la implementación del control de acceso por roles (estudiante, líder, coordinador, administrador), cumpliendo con los requisitos de seguridad del sistema. La integración nativa con Prisma permite mantener coherencia tipada entre la lógica de negocio y la capa de persistencia. Además, NestJS comparte TypeScript con el frontend, lo que permite al equipo trabajar con un único lenguaje en todo el stack, reduciendo la carga cognitiva durante el desarrollo.

Desventajas orientadas al proyecto: NestJS posee una curva de aprendizaje más elevada que frameworks minimalistas como Express.js, especialmente en lo referente a su sistema de decoradores, inyección de dependencias y estructura modular. Para un equipo que no ha trabajado previamente con este framework, la configuración inicial puede requerir una inversión de tiempo considerable.

Express.js

Express.js es el framework minimalista más utilizado dentro del ecosistema Node.js. Proporciona una capa ligera para el manejo de rutas HTTP y middleware, permitiendo al desarrollador configurar la arquitectura de la aplicación con total libertad.

Ventajas orientadas al proyecto: Express.js ofrece la curva de aprendizaje más baja entre las alternativas evaluadas. Su flexibilidad permitiría al equipo comenzar a implementar endpoints rápidamente sin necesidad de aprender patrones arquitectónicos complejos. La cantidad de tutoriales, ejemplos y recursos disponibles es la más amplia dentro del ecosistema Node.js.

Desventajas orientadas al proyecto: Express.js no impone ninguna estructura organizativa, lo que en un proyecto con 10 módulos funcionales, +30 entidades y +17 requisitos funcionales como el propuesto, podría derivar en un código desorganizado y difícil de mantener conforme crece el sistema. No incluye inyección de dependencias, sistema de módulos, ni interceptores de forma nativa, lo que obligaría al equipo a implementar o integrar manualmente estas funcionalidades. La ausencia de una estructura estándar dificultaría además la coherencia entre los diferentes módulos del backend.

Fastify

Fastify es un framework para Node.js enfocado en rendimiento y baja latencia. Ofrece un sistema de plugins para la extensión de funcionalidades y validación de esquemas integrada.

Ventajas orientadas al proyecto: Fastify presenta un rendimiento superior al de Express.js en operaciones de lectura intensiva, lo cual beneficiaría las consultas frecuentes al feed de proyectos y al tablero de tareas. Su sistema de validación de esquemas integrado podría contribuir a la integridad de los datos recibidos en los endpoints del sistema.

Desventajas orientadas al proyecto: Fastify no ofrece una estructura modular comparable a la de NestJS. Para un sistema con múltiples dominios funcionales interconectados, la organización del código dependería enteramente de las convenciones definidas por el equipo, incrementando el riesgo de inconsistencias. Su ecosistema de plugins es menor que el de Express.js y NestJS, y la integración con Prisma y mecanismos de autorización basados en roles requeriría configuración manual adicional. La comunidad y documentación disponible, aunque creciente, es significativamente menor que la de NestJS.

Spring Boot

Spring Boot es un framework backend basado en Java que ofrece una arquitectura empresarial robusta con inyección de dependencias, seguridad integrada y un ecosistema maduro para aplicaciones de gran escala.

Ventajas orientadas al proyecto: Spring Boot proporciona un nivel de madurez y robustez superior para aplicaciones empresariales. Su módulo Spring Security ofrece mecanismos avanzados de autenticación y autorización que podrían cubrir de forma integral los requisitos de seguridad del sistema. La estricta tipificación de Java contribuye a la detección temprana de errores.

Desventajas orientadas al proyecto: La adopción de Spring Boot implicaría introducir un lenguaje adicional (Java) en el stack tecnológico, rompiendo la uniformidad de TypeScript entre frontend y backend. Esto incrementaría la carga cognitiva del equipo y dificultaría el intercambio de validaciones y modelos entre capas. El tiempo de configuración inicial y la complejidad del ecosistema de Spring son considerablemente mayores que los de NestJS, lo cual representa un riesgo dentro del cronograma limitado del proyecto académico. Además, el consumo de recursos de una aplicación Java es típicamente mayor, lo que entraría en conflicto con las limitaciones de infraestructura disponibles para el despliegue del sistema.

Tras evaluar las cuatro alternativas, se seleccionó NestJS como framework backend del sistema. Esta decisión responde a su arquitectura modular que refleja directamente la estructura funcional del proyecto, su compatibilidad nativa con TypeScript que permite mantener un stack unificado, su sistema integrado de guards e interceptores que facilita la implementación de los requisitos de seguridad y auditoría, y su integración fluida con Prisma como ORM. Si bien NestJS presenta una curva de aprendizaje mayor que Express.js, los beneficios en términos de organización, mantenibilidad y escalabilidad del código justifican la inversión inicial de aprendizaje, especialmente considerando que el sistema contempla múltiples módulos funcionales interconectados que requieren una estructura coherente y bien definida.

Selección del Gestor de Base de Datos

Comparación PostgreSQL vs MySQL vs MariaDB vs SQL Server y justificación final.

El sistema propuesto requiere una base de datos relacional robusta que permita mantener integridad referencial, consistencia transaccional y relaciones complejas entre entidades como usuarios, proyectos, tareas, postulaciones y validaciones institucionales. Por ello, se evaluaron cuatro alternativas ampliamente utilizadas dentro del paradigma relacional:

- PostgreSQL
- MySQL
- MariaDB
- Microsoft SQL Server

PostgreSQL

PostgreSQL es un sistema de gestión de bases de datos relacional orientado a objetos que destaca por su cumplimiento estricto del estándar SQL, soporte completo de transacciones ACID y manejo avanzado de consultas complejas.

Según la Stack Overflow Developer Survey 2025, PostgreSQL presenta un 55.6% de adopción global entre desarrolladores y un 58.2% de adopción entre desarrolladores profesionales, superando significativamente a MySQL por aproximadamente 18.6 puntos porcentuales (Stack Overflow, 2025).

Además, PostgreSQL ha liderado durante tres años consecutivos las categorías de “más usada”, “más admirada” y “más deseada” dentro del ecosistema de bases de datos relacionales, mostrando un crecimiento sostenido en adopción (Stack Overflow, 2025).

En el ranking de DB-Engines, PostgreSQL mantiene uno de los mayores índices de crecimiento histórico, alcanzando un puntaje superior a 650 puntos, posicionándose consistentemente entre los sistemas de bases de datos más influyentes del mundo (DB-Engines, 2025).

MySQL

MySQL es una base de datos relacional ampliamente adoptada y tradicionalmente utilizada en aplicaciones web. Ofrece buen rendimiento y facilidad de configuración. Sin embargo, presenta limitaciones en comparación con PostgreSQL en cuanto a cumplimiento estricto del estándar SQL y características avanzadas de extensibilidad.

Aunque sigue siendo popular, su crecimiento en adopción ha sido menor en comparación con PostgreSQL en los últimos años (Stack Overflow, 2025).

MariaDB

MariaDB surge como una bifurcación de MySQL, buscando mantener compatibilidad y mejoras en rendimiento. Si bien ofrece características similares a MySQL, su adopción es considerablemente menor que la de PostgreSQL en el ámbito profesional.

En entornos empresariales de gran escala, MariaDB no presenta el mismo nivel de crecimiento ni posicionamiento estratégico observado en PostgreSQL.

Microsoft SQL Server

SQL Server es una solución robusta orientada principalmente a entornos empresariales que trabajan dentro del ecosistema Microsoft. Aunque ofrece alto rendimiento y herramientas avanzadas de administración, implica mayores costos de licenciamiento en comparación con alternativas open-source.

Dado que el proyecto busca escalabilidad institucional sin dependencia de ecosistemas propietarios, SQL Server no representa la opción más adecuada dentro del contexto universitario del sistema.

Justificación Final

La elección de PostgreSQL se fundamenta en criterios técnicos, estadísticos y estratégicos:

- Mayor adopción global y profesional (Stack Overflow, 2025).
- Crecimiento sostenido y liderazgo en métricas de uso.
- Cumplimiento estricto del estándar SQL.
- Soporte avanzado de transacciones ACID.
- Excelente manejo de relaciones complejas y claves foráneas.
- Amplia adopción por proveedores líderes en infraestructura y servicios en la nube.

Adicionalmente, PostgreSQL se integra de manera óptima con Prisma como ORM y con el stack tecnológico basado en TypeScript, facilitando una arquitectura limpia y mantenible.

Considerando la naturaleza altamente relacional del sistema y la necesidad de consistencia en procesos académicos como validación de horas y auditoría de acciones, PostgreSQL representa la opción más adecuada para garantizar integridad, rendimiento y escalabilidad futura.

Arquitectura General del Sistema

El sistema se diseñará bajo un enfoque de arquitectura por capas inspirado en los principios de Clean Architecture, con el objetivo de garantizar separación de responsabilidades, mantenibilidad, escalabilidad y facilidad de prueba.

La arquitectura se divide en cuatro capas principales:

1. Capa de Presentación
2. Capa de Aplicación
3. Capa de Dominio
4. Capa de Infraestructura

Capa de Presentación

Esta capa corresponde al frontend desarrollado con Next.js y React.

Es responsable de:

- Renderizar la interfaz de usuario.
- Gestionar interacciones del usuario.
- Validar datos preliminarmente con Zod.
- Manejar estado global con Zustand.
- Gestionar datos remotos mediante React Query.

El frontend no contiene lógica de negocio compleja.

Su función es consumir los servicios expuestos por el backend a través de una API REST segura.

Capa de Aplicación

Esta capa está implementada en el backend mediante NestJS.

Es responsable de:

- Orquestar los casos de uso del sistema.
- Aplicar reglas de negocio.
- Gestionar autenticación y autorización.
- Coordinar operaciones entre dominio e infraestructura.

En esta capa se encuentran los controladores y servicios que estructuran el flujo del sistema.

Capa de Dominio

Contiene las entidades centrales del sistema:

- Usuario
- Proyecto
- Postulación
- Tarea
- Validación

Aquí se definen las reglas de negocio independientes de la tecnología utilizada. Esta separación permite que la lógica principal del sistema no dependa directamente de frameworks específicos.

Capa de Infraestructura

Esta capa gestiona la persistencia de datos y comunicación con la base de datos.

Está compuesta por:

- PostgreSQL como gestor relacional.
- Prisma como ORM para mapear entidades del dominio a tablas de base de datos.

Prisma permite traducir objetos TypeScript en consultas SQL seguras, manteniendo integridad referencial y consistencia transaccional.

Interacción entre Capas

El flujo general del sistema funciona de la siguiente manera:

1. El usuario interactúa con la interfaz en el frontend (Next.js).
2. El frontend envía solicitudes HTTP al backend mediante API REST.
3. El backend (NestJS) recibe la solicitud y ejecuta el caso de uso correspondiente.
4. Si se requiere persistencia, el servicio utiliza Prisma.
5. Prisma ejecuta la consulta en PostgreSQL.
6. La respuesta viaja nuevamente hacia el frontend para ser renderizada.

Este flujo garantiza:

- Separación clara de responsabilidades.
- Bajo acoplamiento entre capas.
- Mayor mantenibilidad.
- Escalabilidad futura.

Distribución de Tecnologías por Capa Arquitectónica

Capa Arquitectónica	Tecnología / Herramienta	Responsabilidad dentro del Sistema
Presentación	Next.js	Framework frontend para renderizado híbrido (SSR/CSR) y estructura de aplicación web
Presentación	React	Construcción de componentes reutilizables y gestión de interfaz de usuario
Presentación	TypeScript	Tipado estático para mayor seguridad y mantenibilidad del código
Presentación	Tailwind CSS	Diseño responsivo y estilizado consistente mediante enfoque utility-first
Presentación	shadcn/ui	Componentes UI reutilizables y accesibles
Presentación	React Query (TanStack Query)	Gestión de datos asincrónicos y cacheo de respuestas del backend
Presentación	Zustand	Manejo de estado global ligero en el frontend
Presentación / Aplicación	Zod	Validación estructurada de datos en frontend y backend
Aplicación	NestJS	Framework backend para estructuración modular y manejo de casos de uso
Aplicación	TypeScript	Tipado fuerte para servicios, controladores y lógica de negocio
Dominio	Entidades del sistema	Definición de reglas de negocio independientes de la infraestructura
Infraestructura	Prisma	ORM para mapeo objeto-relacional y gestión segura de consultas
Infraestructura	PostgreSQL	Sistema de gestión de base de datos relacional con soporte ACID

Estimación de las dimensiones del sistema

Cantidad de usuarios probables

La Universidad del Valle de Guatemala cuenta con una población estudiantil activa de aproximadamente 4,000 a 5,000 estudiantes. Considerando que no todos los estudiantes participarían activamente en proyectos interdisciplinarios, se estima que entre 800 y 1,500 usuarios se registrarían en la plataforma durante el primer año de operación. A esta cifra se suman los líderes de asociaciones estudiantiles, estimados en un rango de 60 a 150 usuarios considerando que la universidad cuenta con aproximadamente 30 a 50 asociaciones activas con 2 a 3 líderes cada una, así como entre 10 y 20 coordinadores académicos vinculados a la validación institucional de proyectos. En un horizonte de dos a tres años, el sistema podría alcanzar un universo total de entre 1,500 y 2,000 usuarios registrados.

Usuarios concurrentes promedio

En el contexto universitario, los patrones de uso del sistema estarían concentrados en franjas horarias específicas: entre las 10:00 y las 14:00 horas durante los recesos entre clases, y entre las 19:00 y las 22:00 horas durante la jornada vespertina y nocturna. Se estima que en horario regular la plataforma tendría entre 50 y 150 usuarios trabajando simultáneamente. Durante eventos puntuales, como la publicación de nuevos proyectos o el cierre de períodos de postulación, podrían registrarse picos de hasta 300 usuarios concurrentes. Esta estimación resulta relevante para dimensionar la capacidad de procesamiento y respuesta del servidor.

Tiempo promedio de respuesta

Se establece como objetivo que las operaciones de lectura, tales como la exploración de proyectos, la consulta de detalle de un proyecto y la visualización del tablero de tareas, presenten un tiempo de respuesta inferior a 4 segundos desde la perspectiva del usuario final. Las operaciones de escritura, como el envío de postulaciones, la creación de tareas y la carga de comentarios, deberán completarse en un tiempo máximo de 5 segundos. Para operaciones de mayor carga computacional, como la generación de certificados en formato PDF, se considera aceptable un tiempo de respuesta de hasta 7 a 10 segundos. Estos tiempos responden a las limitaciones de infraestructura disponibles para el proyecto, considerando que el sistema será desplegado en un entorno con recursos de cómputo restringidos propios de un contexto académico universitario.

Escalabilidad y tamaño estimado de almacenamiento

Se proyecta que cada semestre se publicará entre 20 y 40 proyectos dentro de la plataforma. Cada proyecto generará información asociada en forma de tareas, postulaciones, participaciones, revisiones de evidencia, registros de horas y evidencias digitales. La base de datos relacional, sin considerar archivos binarios, no superaría los 500 MB durante los primeros dos a tres años de operación.

El componente de mayor consumo de almacenamiento corresponde a las evidencias digitales. Estimando un peso promedio de 5 MB por evidencia y una generación de entre 30 y 50 evidencias por proyecto, cada proyecto representaría entre 150 y 250 MB en archivos adjuntos. En un ciclo anual de operación, esto equivaldría a un rango de 6 a 20 GB destinados exclusivamente al almacenamiento de evidencias. Considerando el crecimiento orgánico del sistema durante un horizonte de tres años, se estima un requerimiento total de almacenamiento de entre 25 y 50 GB.

Disponibilidad de la información

Dado que el sistema opera en un contexto académico y no constituye un sistema de misión crítica, se establece como objetivo una disponibilidad mínima del 99%, lo cual equivale a un tiempo máximo de inactividad de aproximadamente 3.6 días al año o 7 horas por mes. Se recomienda que las ventanas de mantenimiento programado se ejecuten fuera de los horarios de uso pico, preferiblemente durante la madrugada o los fines de semana, para minimizar el impacto sobre los usuarios.

Confiabilidad, tolerancia a fallos y replicación de la información

Considerando que el sistema gestionará información de carácter institucional, incluyendo validaciones de horas académicas y emisión de certificados oficiales, la pérdida de datos representaría un impacto significativo tanto para los estudiantes como para los coordinadores académicos. Por esta razón, se establece un esquema de respaldos automáticos diarios con una retención mínima de 7 días, tal como se especifica en los requisitos no funcionales del sistema.

El objetivo de tiempo de recuperación (RTO) se define en un máximo de 1 a 2 horas ante una falla del servidor principal, y el punto de recuperación objetivo (RPO) no deberá exceder las 24 horas, lo que implica que en el peor escenario se perdería como máximo un día de información. Para el alcance actual del proyecto no se contempla la implementación de replicación en tiempo real ni mecanismos de failover automático; sin embargo, se considera indispensable contar con un plan documentado de recuperación ante desastres que garantice la continuidad operativa del sistema.

Planificación

Cronograma del proyecto

Semana	Fecha	Responsable	Tarea	Entregable
Semana 1	1-2 Mar	Vernel Hernández	Definir contexto y alcance del sistema	Sección de descripción y alcance del documento
	1-2 Mar	Saul Castillo	Formalizar requisitos funcionales del sistema	Lista numerada de requisitos funcionales
	3-4 Mar	Derek Coronado	Definir lista final de clases preliminares	Listado de clases con justificación
	3-4 Mar	Samuel Robledo	Consolidar versiones de prototipos y crear tabla de cambios	Tabla de versiones con cambios y motivos
	3-4 Mar	Ángel Sanabria	Revisión de la ideación hecha por Vernel Hernández	Documento de revisión con observaciones
	5-6 Mar	Vernel Hernández	Diseñar clases persistentes y definir llaves primarias y foráneas	Esquema de entidades con PK y FK definidas
	5-6 Mar	Saul Castillo	Enlazar requisitos con historias de usuario y clasificar requisitos no funcionales	Tabla de trazabilidad HU-RF y lista de RNF
	5-7 Mar	Derek Coronado	Crear Diagrama de Clases de Análisis (UML)	Diagrama UML con atributos y relaciones
Semana 2	7-8 Mar	Vernel Hernández	Definir dependencias funcionales y proceso de normalización (1FN, 2FN, 3FN, FNBC)	Documento con evidencia paso a paso de normalización
	7-8 Mar	Derek Coronado	Describir responsabilidades de cada clase y elaborar Diagrama de Paquetes	Descripción de clases y diagrama de paquetes
	7-8 Mar	Saul Castillo	Redactar sección formal de requisitos del documento técnico	Sección completa de requisitos formateada
	8-9 Mar	Ángel Sanabria	Realización del Diagrama Entidad-Relación (DER)	Modelo conceptual ER completo con cardinalidades
	9-10 Mar	Derek Coronado	Estimar crecimiento de datos del sistema	Sección de estimación de dimensiones

	9-10 Mar	Samuel Robledo	Diseñar la presentación para el corte	Archivo de presentación (borrador)
	10-12 Mar	Ángel Sanabria	Realización del Script DDL en SQL	Script DDL ejecutable sin errores
	10-12 Mar	Vernel Hernández	Elaborar cronograma del proyecto	Tabla de cronograma con responsables y fechas
	12-13 Mar	Saul Castillo	Realizar el guion de la presentación	Guion escrito para la presentación
	13-14 Mar	Samuel Robledo	Revisar el guion de la presentación y ajustes finales	Guion revisado y presentación final

Distribución de carga por integrante

Integrante	Tareas	Responsabilidades principales
Ángel Sanabria	5	Revisión de ideación, Diagrama Entidad-Relación (DER), Script DDL en SQL, Revisión del corte, Selección de tecnologías
Saul Castillo	4	Requisitos funcionales, trazabilidad HU-RF, requisitos no funcionales, guion de presentación
Vernel Hernández	4	Contexto y alcance, clases persistentes con PK/FK, normalización (1FN-FNBC), cronograma
Derek Coronado	4	Clases preliminares, Diagrama de Clases UML, Diagrama de Paquetes, estimación de datos
Samuel Robledo	3	Consolidación de prototipos, diseño de presentación, revisión de guion

Conclusiones del equipo

El desarrollo del tercer corte del proyecto representó un avance significativo en la madurez técnica y organizativa del equipo de trabajo. A lo largo de esta etapa, se lograron formalizar los requisitos funcionales y no funcionales del sistema, elaborar el modelo de clases preliminar con sus respectivas relaciones, diseñar el diagrama entidad-relación de la base de datos y fundamentar la selección tecnológica mediante análisis comparativos detallados.

Entre los aspectos positivos del desempeño del equipo se destacan la capacidad de mantener coherencia entre las diferentes secciones del documento, asegurando trazabilidad desde las historias de usuario hasta los requisitos funcionales, las clases del dominio y los módulos del backend. Asimismo, la definición de requisitos no funcionales con criterios de medición específicos refleja un nivel de rigor

que permitirá evaluar objetivamente el sistema durante la fase de pruebas. La selección tecnológica se realizó de manera fundamentada, evaluando múltiples alternativas para cada capa de la arquitectura y justificando las decisiones con base en criterios técnicos y en las necesidades concretas del proyecto.

No obstante, se identifican áreas de mejora para las siguientes etapas del desarrollo. La validación de prototipos con usuarios, si bien se realizó y se documentaron sus resultados, podría beneficiarse de un proceso más estructurado que incluya métricas cuantitativas sobre la interacción de los usuarios con la plataforma. Adicionalmente, la gestión del tiempo del equipo requiere mayor disciplina en el registro sistemático de actividades, lo cual será fundamental para optimizar la distribución de carga de trabajo durante la fase de implementación. Finalmente, la coordinación entre los integrantes del equipo durante la elaboración simultánea de diferentes secciones del documento presentó desafíos que podrían mitigarse mediante reuniones de sincronización más frecuentes y la definición anticipada de estándares de redacción y formato.

El equipo reconoce que la transición hacia la fase de implementación requerirá traducir la documentación generada en este corte en código funcional, lo cual exigirá una planificación más granular de las tareas técnicas y una comunicación constante para asegurar que la arquitectura diseñada se refleje fielmente en el producto final.

Conclusiones

En la sección de Design Thinking se documentaron las iteraciones realizadas sobre los prototipos del sistema, incorporando la retroalimentación obtenida durante las validaciones con usuarios potenciales. Este proceso permitió refinar la interfaz de la plataforma e identificar funcionalidades clave como el tablero de tareas tipo Kanban y la visualización de progreso individual, las cuales fueron incorporadas como requisitos funcionales del sistema.

En la sección de Análisis se especificaron 17 historias de usuario agrupadas por perfil de actor, 17 requisitos funcionales con trazabilidad explícita hacia las historias de usuario, y un modelo de clases preliminar compuesto por 14 entidades de dominio con sus respectivos atributos, operaciones y relaciones. El diagrama de paquetes refleja la organización modular del sistema en cuatro capas arquitectónicas con principios de alta cohesión y bajo acoplamiento.

En la sección de Persistencia se elaboró el diagrama entidad-relación de la base de datos relacional, representando las entidades, atributos, claves primarias, claves foráneas y las relaciones entre las tablas que soportarán el almacenamiento del sistema. Se realizó una comparación detallada entre cuatro gestores de bases de datos relacionales, seleccionando PostgreSQL como la opción más adecuada con base en criterios de adopción profesional, cumplimiento del estándar SQL, soporte de transacciones ACID e integración con el stack tecnológico definido.

En la sección de Diseño se definieron los requisitos no funcionales del sistema con criterios de medición específicos y comprobables, abarcando categorías de apariencia, usabilidad, rendimiento, soporte, portabilidad, seguridad, confiabilidad y restricciones de implementación. Se realizaron estimaciones de las dimensiones del sistema fundamentadas en el contexto institucional de la Universidad del Valle de Guatemala. La selección tecnológica fue respaldada mediante análisis comparativos para cada capa de la arquitectura: Next.js frente a Angular, Nuxt y SvelteKit para el frontend; NestJS frente a Express.js,

Fastify y Spring Boot para el backend; y PostgreSQL frente a MySQL, MariaDB y SQL Server para la persistencia de datos.

Referencias

Colin McDonnell. (2025). Zod documentation. https://zod.dev

DB-Engines. (2025). DB-Engines ranking. https://db-engines.com/en/ranking

Martin, R. C. (2017). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.

Meta. (2025). React documentation. https://react.dev

NestJS. (2025). NestJS documentation. https://docs.nestjs.com

npmtrends.com (s. f.). React – npm trends. https://npmtrends.com/react

npmtrends.com (s. f.). React Query vs Zustand – npm trends. https://npmtrends.com/react-query-vs-zustand

npmtrends.com. (s. f.). Zustand – npm trends. https://npmtrends.com/zustand

npm-compare.com (s. f.). shadcn – npm compare. https://npm-compare.com/shadcn

Prisma. (2025). Prisma documentation. https://www.prisma.io/docs

Stack Overflow. (2025). 2025 Developer Survey: Technology results. https://survey.stackoverflow.co/2025/technology/

Tailwind Labs. (2025). Tailwind CSS documentation. https://tailwindcss.com/docs

TanStack. (2025). TanStack Query documentation. https://tanstack.com/query/latest

Vercel. (2025). Next.js documentation. https://nextjs.org/docs